

TDT4125 Algoritmekonstruksjon

Eksamen, 12. mai 2026, 15:00–19:00

Faglig kontakt Magnus Lie Hetland

Hjelpemiddelkode E

Løsningsforslag

Løsningsforslagene i rødt nedenfor er *eksempler* på svar som vil gi full uttelling. Det vil ofte være helt akseptabelt med mange andre, beslektede svar, spesielt der det bes om en forklaring eller lignende. Om du svarte noe litt annet, betyr ikke det nødvendigvis at du svarte feil!

- 1 De lineære programmene P og D er duale av hverandre. Hva kan du si om forholdet mellom dem?

Diskuter gjerne situasjonen både for optimale og ikke-optimale løsninger, og tilfeller der det ikke finnes noen optimal løsning.

Ett av problemene er et maksimeringsproblem og det andre, et minimeringsproblem. Enhver gyldig løsning på maksimeringsproblemet har en verdi som er mindre eller lik enhver gyldig løsning på minimeringsproblemet.

Det betyr også at hvis det ene problemet er ubegrenset (*unbounded*), og altså kan få vilkårlig gode løsninger, så kan ikke det andre problemet ha *noen* gyldige løsninger (dvs., det må være ugyldig, eller *unfeasible*). Det er også mulig at *ingen* av problemene har gyldige løsninger.

Hvis det ene problemet har en optimal løsning, så har begge det, og disse løsningene har samme verdi.

- 2 Hvordan bestemmer vi fortegnet til en dualvariabel? Det vil si, hvordan vet vi om vi skal kreve $y_i \geq 0$, $y_i \leq 0$ eller ingen av delene? Forklar kort.

Her er det en del kombinasjoner, men vi kan f.eks. bruke *The S-O-B Method* for å holde styr på dem, der hver variabel og restriksjon får merkelappen *sensible*, *odd* eller *bizarre*.

Her er $y_i \geq 0$ *sensible*, en ubegrenset y_i er *odd* og $y_i \leq 0$ er *bizarre*.

For maksimering, regnes $\leq$ -restriksjoner som *sensible*, $=$ -restriksjoner som *odd* og $\geq$ -restriksjoner som *bizarre*. For minimering, bytter vi klassifiserin-

gen for $\leq$ og $\geq$.

Prinsippet er så at dualvariebelen skal ha samme klassifisering som tilsvarende restriksjon.

Her godkjennes naturligvis også forklaringer som ikke involverer S-O-B-metoden. Den er kun tatt med i pensum for å gjøre det lettere å huske disse sammenhengene.

- 3 Du har n arbeidsoppgaver som skal utføres, og har m arbeidslag du kan leie inn. Du vet hvor mange personer du trenger for hver oppgave, og for hvert lag vet du hvor mange som er kompetente til å utføre hver av oppgavene. Hvert lag har en pris, og du skal velge ut et sett med lag som gir deg nok kompetente personer til hver arbeidsoppgave, slik at den totale prisen minimeres. Merk at en person kan utføre flere ulike arbeidsoppgaver, uten at det påvirker prisen.

Forklar kort hvordan du ville ha satt opp problemet som et heltallig lineærprogram.

Du trenger ikke bruke matematisk notasjon. En skissepreget beskrivelse er tilstrekkelig, så lenge den er entydig og forståelig.

Problemene kan settes opp på samme måte som for vektet mengdedekke. Eneste endring er at matrisen A ikke lenger er binær, men at a_{ij} inneholder hvor mange i arbeidslag i som er kompetente til å utføre oppgave j . På samme måte har vi ikke lenger $b_j = 1$, men lar b_j være hvor mange som trengs for å utføre oppgave j .

- 4 Forklar hvordan dualbasert avrunding vil fungere for programmet ditt i forrige oppgave, og hvilken ytelsesgaranti/approksimasjonsfaktor α det vil gi.

Relakser det binære programmet til et vanlig lineærprogram. Løs dette, og rund opp primærvariable til 1 ved stram dual (dvs., der dualen er oppfylt ved likhet). Tilsvarende som for mengdedekke, ender vi med forholdet mellom venstre side og høyre side i restriksjonene, altså:

$$\alpha = \max_j \{ \text{antall kompetente til jobb } j / \text{antall som kreves for jobb } j \}$$

Dette følger av at vi har approksimert komplementær slakket.

Mer utdypende: Siden vi kun runder opp ved stram dual, kan ev. avvik fra komplementær slakket bare komme fra slakk i primalrestriksjoner med positive dualvariable. På samme måte som for mengdedekke, kan vi finne en øvre grense ved å anta at alle dualvariable er positive, og så finne største relative slakk, dvs., forholdet mellom verdiene på venstre og høyre side i en primalrestriksjon, igjen kan vi få en øvre grense ved å summere alle koeffisientene i en gitt rad (tilsvarer frekvensen f som brukes for mengdedekke).

Det er også akseptabelt, og strengt tatt egentlig bedre, å runde opp alle

strengt positive primalvariable, siden disse uansett vil ha stram dual, og vi ikke vil ha noen nytte av å runde opp primalvariable med verdi 0 som har stram dual.

- 5 I pensum presenteres Kargers algoritme, en enkel randomisert algoritme for å finne minste snitt i en urettet, uvektet graf. Forklar kort og skissepreget hvordan algoritmen fungerer.

Grunnversjonen av algoritmen feiler med en sannsynlighet på ikke mer enn $1 - 1/n^2$, for n noder. Forklar også hvordan man normalt forbedrer den til å redusere denne sannsynligheten til under $1/e$.

Merk: Det er ikke nødvendig med sannsynlighets-utregninger eller formler. Det holder med en overordnet gjengivelse av hvilke trinn som utføres.

Grunnversjon: Velg én og én kant tilfeldig, og kollaps den. Dvs., slå sammen endenodene og fjern løkker (*self-loops*), men bevar parallelle kanter. (Vi har altså å gjøre med en multigraf underveis i utførelsen.)

Forbedring: Gjenta algoritmen flere ganger, og velg den beste løsningen.

Antall ganger man gjentar algoritmen for å komme under $1/e$ er $n^2/2$, men det kreves ikke at man oppgir dette.

- 6 Når vi bruker maks-flyt til å finne et minste snitt, er det snakk om et s - t -snitt, altså et snitt der kilden s og sluket t ligger på hver sin side. En naturlig utvidelse av Kargers algoritme til dette tilfellet er at man innfører et forbud mot å plassere s og t sammen, uten å gjøre noen ytterligere endringer i algoritmen. Den modifiserte algoritmen vil da produsere et s - t -snitt, men sannsynligheten for å ende opp med det *minste* kan bli eksponentielt liten, som i praksis gjør algoritmen ubrukelig! Forklar, gjerne ved hjelp av et eksempel.

For enkelhets skyld ser vi fortsatt kun på urettede grafer. Du kan også anta at instansen er en multigraf, det vil si, du kan ha flere enn én kant mellom hvert par med noder.

For eksempel: Legg til noder $v_1, v_2, \dots$ med *to* kanter mellom s og v_i og *én* kant mellom v_i og t . For hver i , må vi da velge kanten (v_i, t) for å få riktig svar, men sannsynligheten for det er bare $1/3$. Sannsynligheten for at vi velger riktig *hver gang* (og dermed finne det minimale snittet) synker eksponentielt med antall noder.

- 7 Vi tar normalt utgangspunkt i at grådighet gir optimum over matroider dersom vi maksimerer over ikke-negative vektfunksjoner. Gi et kort argument for at dette medfører at en grådig strategi også fungerer for *minimering* over ikke-negative vektfunksjoner.

Merk at løsningene fortsatt skal være maksimale, dvs., ikke utvidbare.

Vi kan begynne med å skifte fortegn på vektfunksjonen. Siden alle maksimale løsninger (basiser) i en matroide har like mange elementer, kan vi så øke verdien til hvert element i grunnmengden med samme verdi, som er stor nok til at vektfunksjonen igjen blir ikke-negativ. Den opprinnelige grådige algoritmen vil finne en maksimal løsning med størst mulig vekt for denne transformerte instansen. Det tilsvarer en maksimal løsning med minst mulig verdi for den opprinnelige instansen, og algoritmen tilsvarer også en grådig strategi på den opprinnelige instansen, der vi hele tiden velger letteste heller enn tyngste element.

Se delkapittel 5.5 i kap. 7 i kompendiet.

- 8 En *uniform* matroide av rang r er en matroide der alle delmengder av grunnmengden med opptil r elementer er uavhengige.

Anta at du har et uavhengighetssystem som er snittet av n matroider, der m av matroidene er uniforme. Hvilken ytelsesgaranti α vil approksimasjon ved grådighet gi for dette uavhengighetssystemet? Forklar.

Første innsikt: Snittet av to uniforme matroider er bare en ny uniform matroide med den minste rangen av de to. Vi har altså å gjøre med snittet av $n - m$ vilkårlige matroider og én uniform matroide.

Andre innsikt: Snittet av en matroide $M = (E, S)$ og en uniform matroide U med rang r er en matroide. For enhver $A \subseteq E$ vil alle basiser (ikke utvidbare delmengder av A) i M ha like mange elementer; det er én av definisjonene på hva det vil si å være en matroide. Dersom dette antallet ikke er mer enn r , vil basisene bevares. Dersom det er høyere enn r , vil alle delmengder av basisene med opptil r elementer bevares, og vi ender fortsatt med at alle basiser har like mange elementer, og altså fortsatt ha en matroide.

Om vi erstatter én av de $n - m$ vilkårlige matroidene med snittet av denne matroiden og den uniforme matroiden, sitter vi altså igjen med $n - m$ matroider som vi så tar snittet av. Og for snittet av k matroider, gir grådighet $\alpha = 1/k$, så vi får en ytelsesgaranti på $\alpha = 1/k$.

Siden alle de uniforme matroidene kan ha $r = |E|$, kan vi ikke få noen bedre garanti enn dette, generelt.

- 9 Hva er forskjellen mellom FPT og XP? Forklar kort.

Et parametrisert problem er *slice-wise polynomial* (XP) hvis det finnes en algoritme $\mathcal{A}$ som har polynomisk kjøretid for enhver gitt parameter k . Det er *fixed-parameter tractable* (FPT) hvis det kun er konstantfaktoren som endres med k , dvs., kjøretiden er begrenset av samme polynom for alle k .

La det parametriserte problemet være $L \subseteq \Sigma^* \times \mathbb{N}$. Mer formelt krever XP at algoritmen har kjøretid $f(k) \cdot |(x, k)|^g(k)$ for instanser $(x, k) \in L$, mens FPT krever $f(k) \cdot |(x, k)|^c$, der $f, g : \mathbb{N} \rightarrow \mathbb{N}$ er beregnbare funksjoner og c er en konstant. En mer konseptuell beskrivelse er tilstrekkelig for full uttelling.

- 10** For det parametriserte problemet $L \in \Sigma^* \times \mathbb{N}$ har vi en algoritme $\mathcal{A}$ som kan avgjøre $(I, k) \in L$ med kjøretid $8^k \cdot |I|^c$. Forklar kort hvorfor dette betyr at vi har en problemkjerne (*kernel*) med en størrelse på maksimalt $2^k + k$.

For halv uttelling, vis i stedet at størrelsen er maksimalt $8^k + k$.

Vi kan følge en tilsvarende logikk som i Lemma 2.2 (kap. 13) i kompendiet: Utfør opptil $|I|^{c+3}$ trinn av $\mathcal{A}$. Hvis vi får et svar, returnerer vi det. Hvis ikke, kan vi bruke (I, k) som kjerne. $\mathcal{A}$ har ikke terminert, som betyr:

$$T(n) = 8^k \cdot |I|^c > |I|^{c+3} \quad (1)$$

$$8^k > |I|^3 \quad (2)$$

$$2^k > |I| \quad (3)$$

$$2^k + k > |I| + k = \text{size}_{\mathcal{A}}(k) \quad (4)$$

Om vi bruker $c + 1$ heller enn $c + 3$, ender vi med $8^k + k$, som tilsvarer resonnementet i lemma 2.2.

For interesserte: Merk at vi kan bruke (polynomisk) flere operasjoner, og etablere en lavere grense. Det er altså ikke noe spesielt med akkurat $2^k + k$.